



Universidade de Brasília

Departamento de Ciência da Computação

Aritmética Inteira



Principais Arquiteturas Aritméticas

■ Pilha:

- As operações são sempre realizadas com os argumentos na pilha e o resultado é também armazenado na pilha.

■ Acumulador:

- As operações são feitas sobre registradores (incluindo A) e o resultado armazenado em um registrador especial chamado Acumulador (A)

■ Registrador-registrador:

- As operações são feitas sobre registradores e o resultado é armazenado em qualquer registrador.

■ Registrador-memória:

- As operações aritméticas buscam um dos argumentos na memória e armazenam o resultado em um registrador.



Representação Numérica

■ Base binária (base 2)

Símbolos: 0,1

□ Ex.: $124 = 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 1111100_2$

■ Base octal (base 8)

Símbolos: 0,1,2,3,4,5,6,7

□ Ex.: $124 = 1 \times 8^2 + 7 \times 8^1 + 4 \times 8^0 = 174_8$

■ Base decimal (base 10)

Símbolos: 0,1,2,3,4,5,6,7,8,9

□ Ex.: $124 = 1 \times 10^2 + 2 \times 10^1 + 4 \times 10^0 = 124_{10}$

■ Base hexadecimal (base 16)

Símbolos: 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

□ Ex.: $124 = 7 \times 16^1 + 12 \times 16^0 = 7C_{16}$



Aritmética Computacional

- Operações com inteiros
 - ☐ Soma e subtração
 - ☐ Multiplicação e divisão
 - ☐ Tratamento de transbordo (*overflow*)

- Operações com números reais
 - ☐ Representação
 - ☐ operações



Inteiros em Complemento de 2

- Bit mais significativo tem peso negativo

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \dots + x_12^1 + x_02^0$$

- Faixa de representação: -2^{n-1} a $+2^{n-1} - 1$



RISC-V RV32I

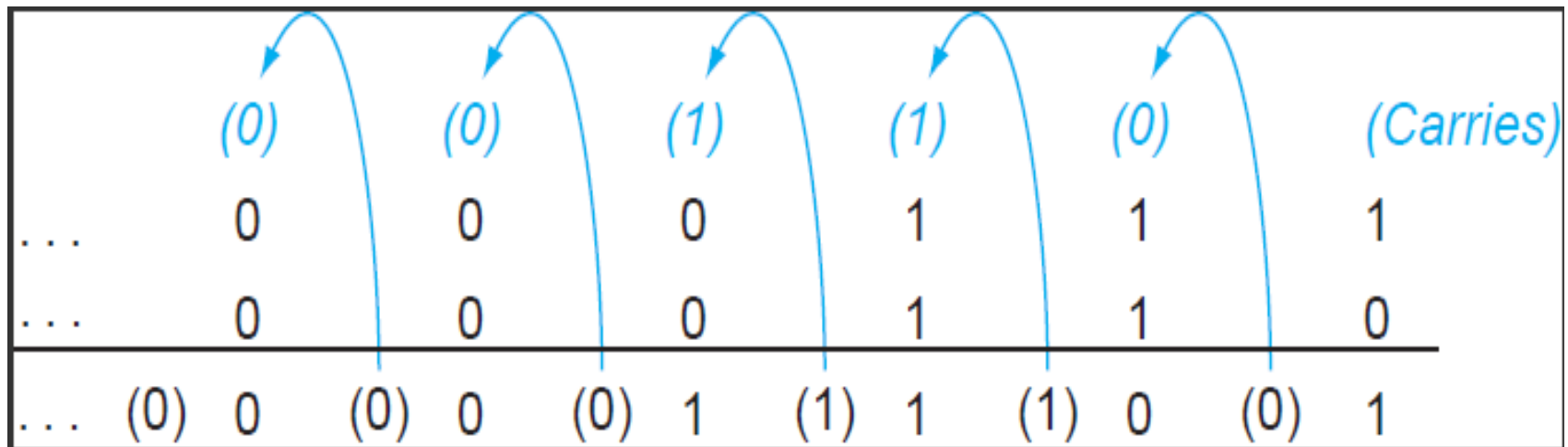
Números de 32 bits com sinal:

0000 0000 0000 0000 0000 0000 0000 0000	$_{b1n}$	=	0_{dec}
0000 0000 0000 0000 0000 0000 0000 0001	$_{b1n}$	=	1_{dec}
0000 0000 0000 0000 0000 0000 0000 0010	$_{b1n}$	=	2_{dec}
...	...		
0111 1111 1111 1111 1111 1111 1111 1101	$_{b1n}$	=	$2.147.483.645_{dec}$
0111 1111 1111 1111 1111 1111 1111 1110	$_{b1n}$	=	$2.147.483.646_{dec}$
0111 1111 1111 1111 1111 1111 1111 1111	$_{b1n}$	=	$2.147.483.647_{dec}$
1000 0000 0000 0000 0000 0000 0000 0000	$_{b1n}$	=	$-2.147.483.648_{dec}$
1000 0000 0000 0000 0000 0000 0000 0001	$_{b1n}$	=	$-2.147.483.647_{dec}$
1000 0000 0000 0000 0000 0000 0000 0010	$_{b1n}$	=	$-2.147.483.646_{dec}$
...	...		
1111 1111 1111 1111 1111 1111 1111 1101	$_{b1n}$	=	-3_{dec}
1111 1111 1111 1111 1111 1111 1111 1110	$_{b1n}$	=	-2_{dec}
1111 1111 1111 1111 1111 1111 1111 1111	$_{b1n}$	=	-1_{dec}



Soma de 2 Números Binários

- Similar ao decimal
- O que não é representado por um dígito é passado ao dígito seguinte, como “vai-um”





Subtração de inteiros

- Negar o segundo operando e somar
- Exemplo: $7 - 6 = 7 + (-6)$

$$\begin{array}{r} +7: 0000 \ 0000 \ \dots \ 0000 \ 0111 \\ -6: 1111 \ 1111 \ \dots \ 1111 \ 1010 \\ \hline +1: 0000 \ 0000 \ \dots \ 0000 \ 0001 \end{array}$$

- *Overflow* se o resultado estiver fora da faixa
 - ☐ POS - NEG = NEG ou
 - ☐ NEG - POS = POS



Detectando *Overflow*

- Ocorre *overflow* (transbordo) se
 - Ao somar dois número de mesmo sinal
 - Sinal do resultado é diferente dos operandos
 - Ao subtrair dois números de sinais diferentes
 - Equivale a somar dois números de mesmo sinal
- Não ocorre *overflow*
 - Ao subtrair dois números de mesmo sinal



Efeitos do *Overflow*

- No RISC-V não se detecta *overflow* na aritmética inteira
 - ☐ O objetivo é simplificar o hardware do processador
 - ☐ Detecção deve ser feita por software
- Obs.: C versus FORTRAN
 - ☐ A linguagem C não prevê a detecção de *overflow* em suas instruções
 - ☐ Fortran prevê a detecção de *overflow* em suas instruções
- MIPS gera exceção

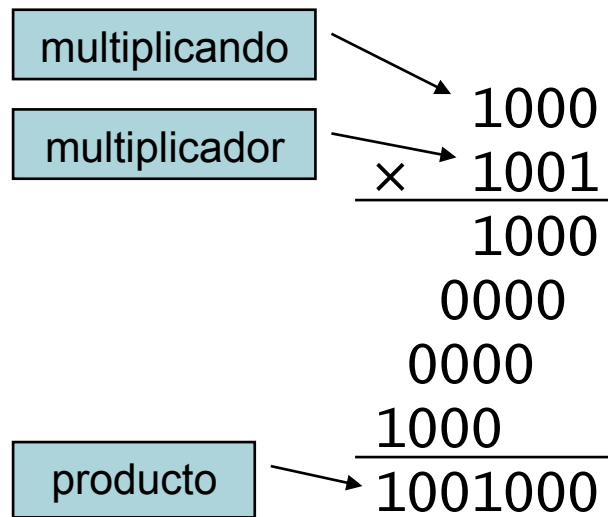


Aritmética para Multimídia

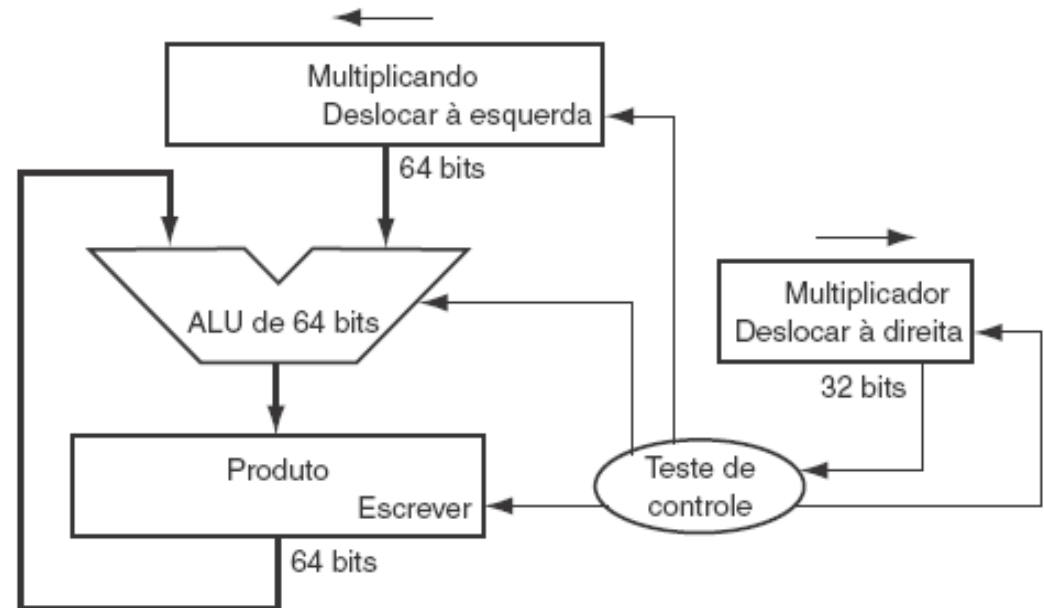
- Processadores gráficos ou de mídia operam com vetores de dados de 8 ou 16 bits
 - Usando um somador de 64 bits, pode-se operar
 - 8 bytes (8 bits), 4 *half words* (16 bits) ou 2 *words* (32 bits)
 - SIMD (*single-instruction, multiple data*)
- Aritmética com saturação
 - No *overflow*, em vez de voltar a zero, o resultado é o valor máximo
 - Saturação em vídeo ou áudio



Multiplicação: Versão Sequencial

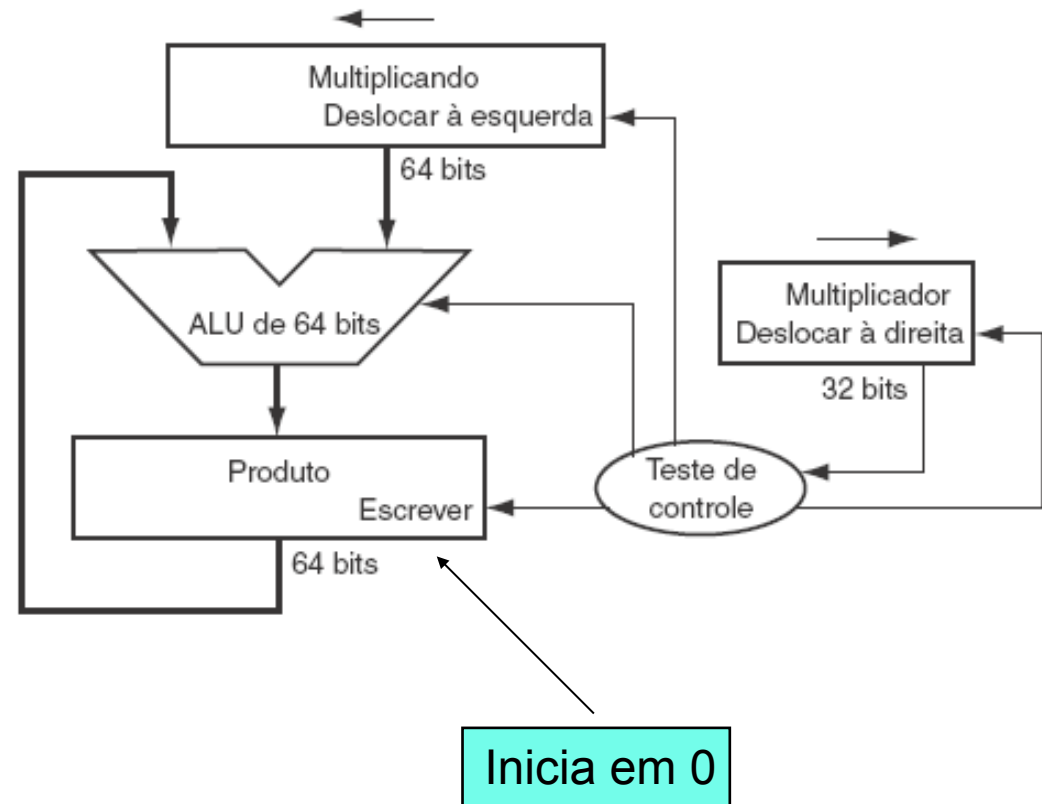
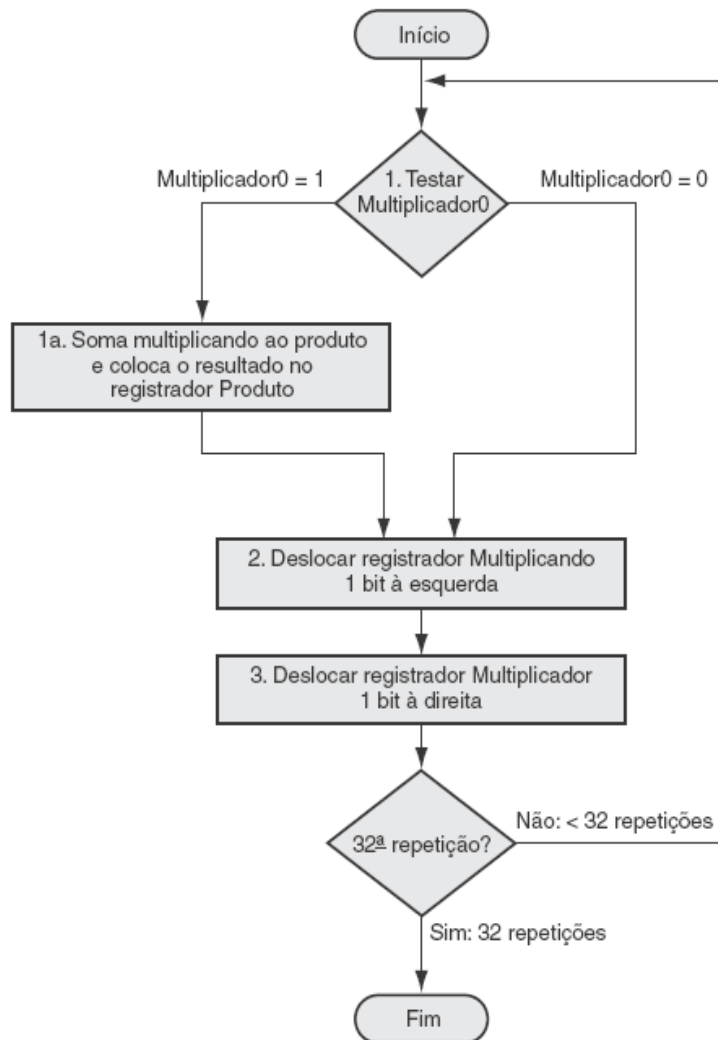


Tamanho do produto
é a soma do
tamanho dos
operandos





Hardware para Multiplicação





Multiplicação - exemplo

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	001 <u>1</u>	0000 0010	0000 0000
1	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0011	0000 0010	0000 0010
	2: Shift left Multiplicand	0011	0000 0100	0000 0010
	3: Shift right Multiplier	000 <u>1</u>	0000 0100	0000 0010
2	1a: $1 \Rightarrow \text{Prod} = \text{Prod} + \text{Mcand}$	0001	0000 0100	0000 0110
	2: Shift left Multiplicand	0001	0000 1000	0000 0110
	3: Shift right Multiplier	000 <u>0</u>	0000 1000	0000 0110
3	1: $0 \Rightarrow$ no operation	0000	0000 1000	0000 0110
	2: Shift left Multiplicand	0000	0001 0000	0000 0110
	3: Shift right Multiplier	000 <u>0</u>	0001 0000	0000 0110
4	1: $0 \Rightarrow$ no operation	0000	0001 0000	0000 0110
	2: Shift left Multiplicand	0000	0010 0000	0000 0110
	3: Shift right Multiplier	0000	0010 0000	0000 0110

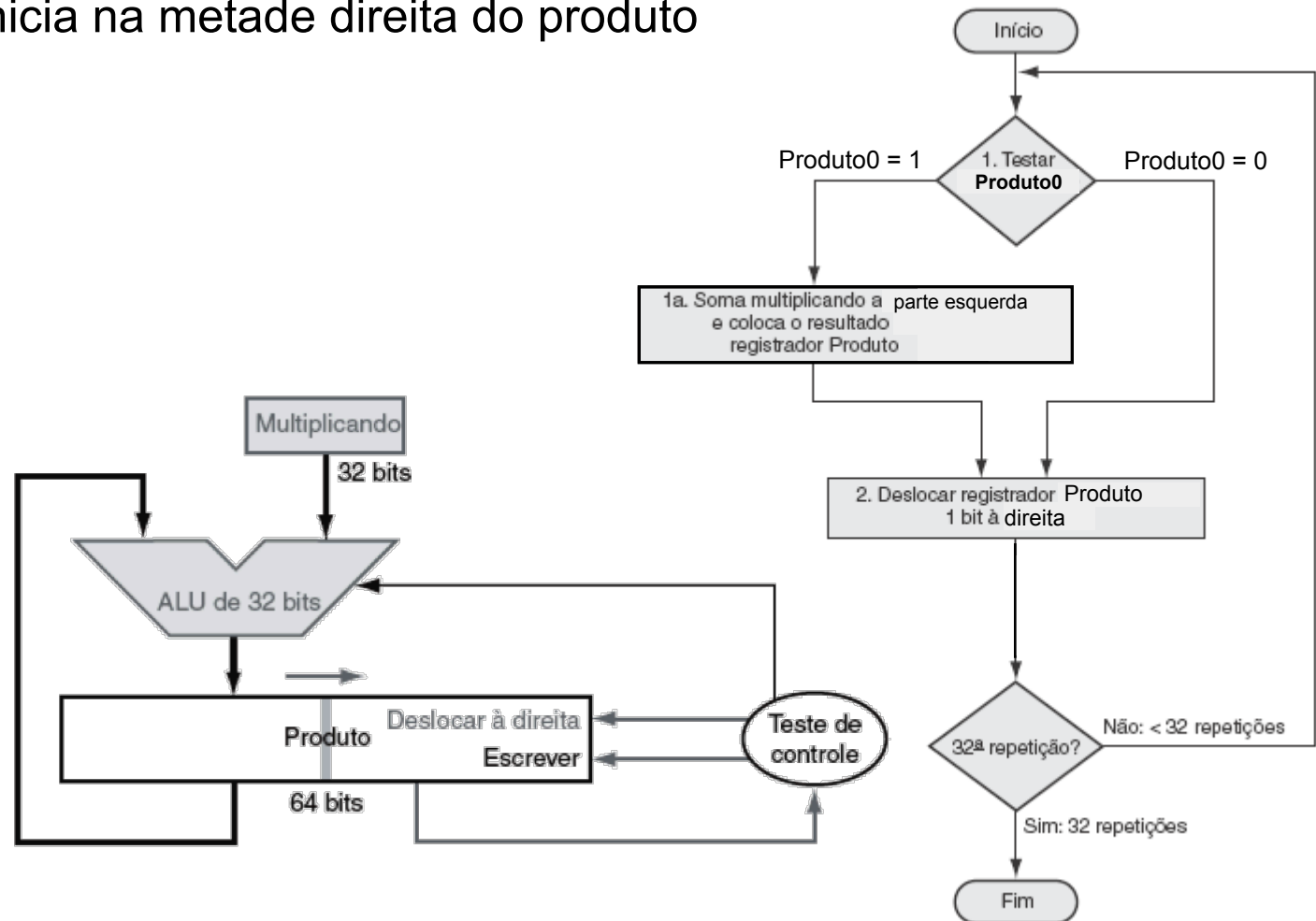


Versão Otimizada

O multiplicador inicia na metade direita do produto

```

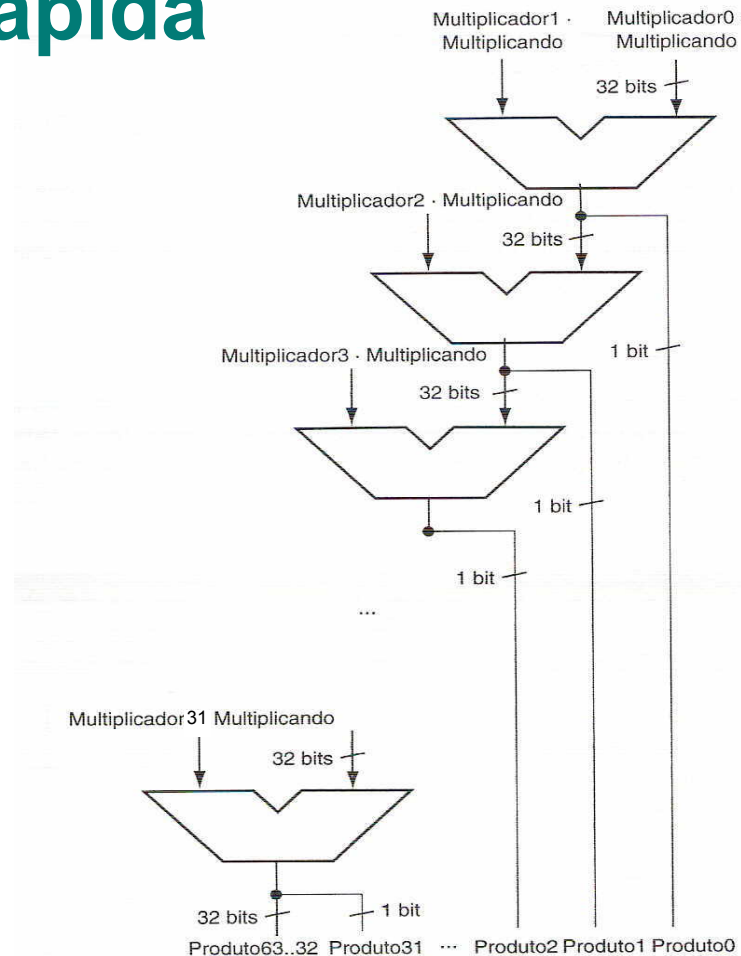
  0010
x 1011
-->>0010
  00010
+ 0010
-->>00110
  000110
+ 0000
-->>000110
  0000110
+ 0010
-->>0010110
  00010110
  
```





Multiplicação: Versão mais Rápida

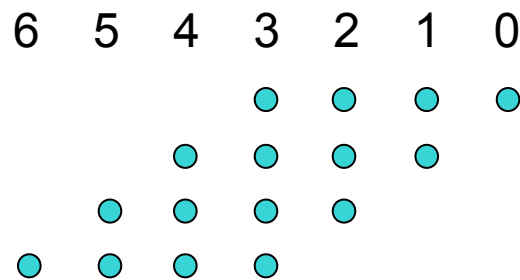
-Maior área em chip



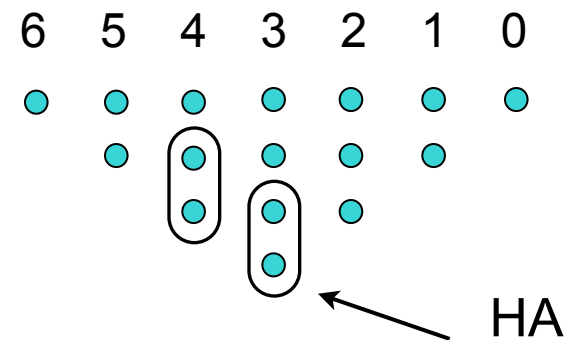


Multiplicador Wallace-Tree (1)

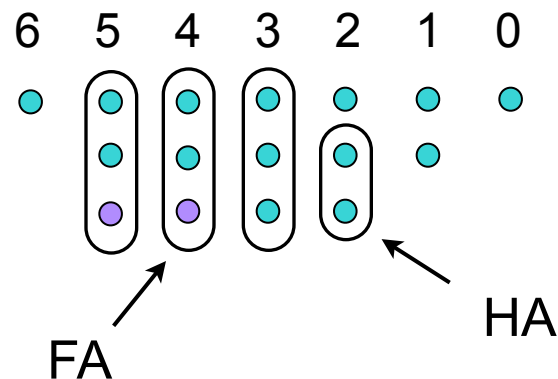
Produtos Parciais



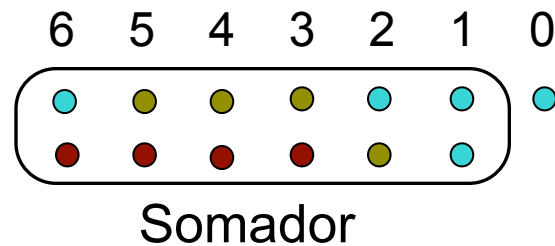
Primeiro Estágio



Segundo Estágio

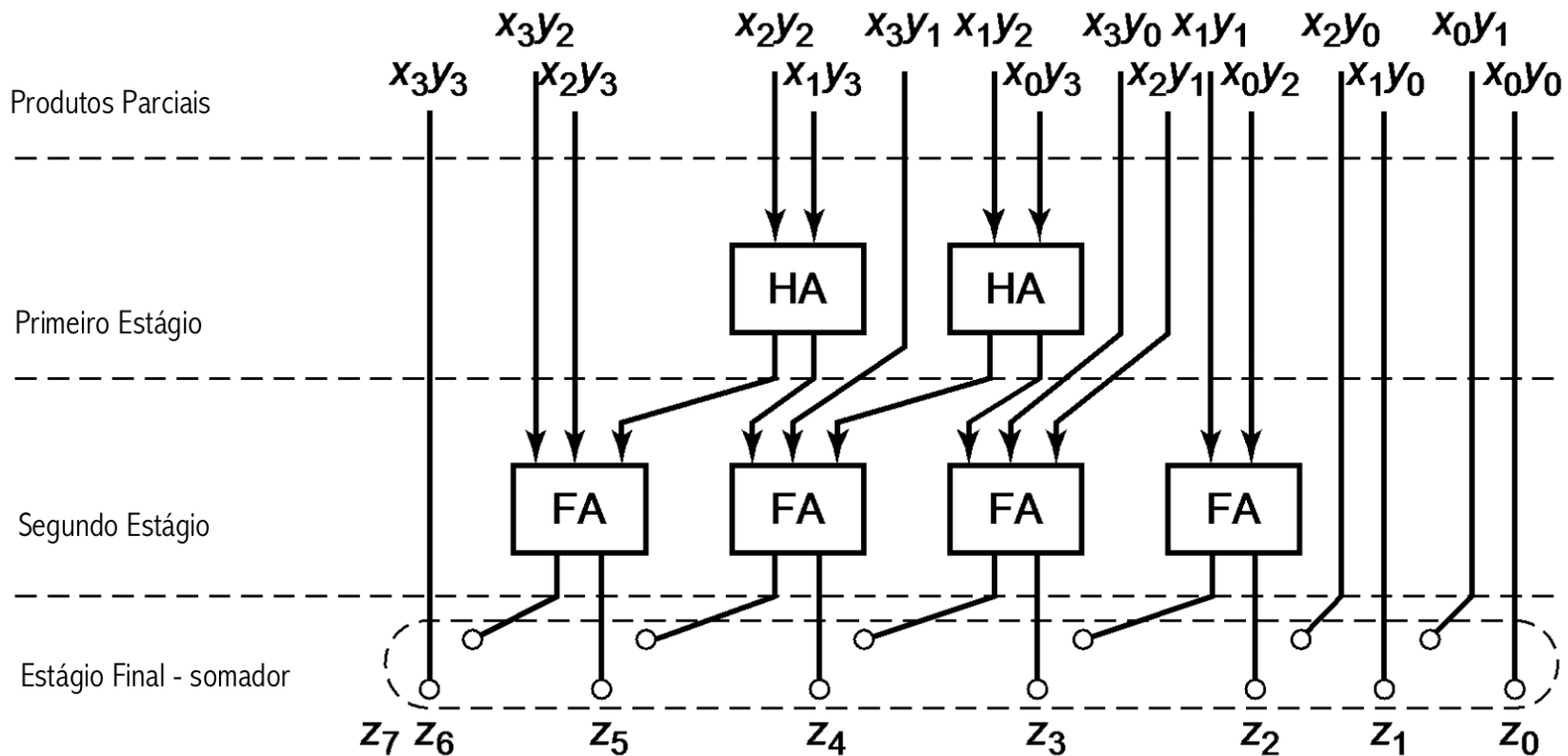


Estágio Final





Multiplicador Wallace-Tree (2)





Multiplicação no RISC-V

■ Quatro instruções

□ mul: multiplica

- Retorna os 32 bits menos significativos do produto

□ mulh: multiplica parte alta

- Retorna os 32 bits mais significativos do produto, assumindo que os operandos tem sinal

□ mulhu: multiplica parte alta sem sinal

- Retorna os 32 bits mais significativos do produto, assumindo que os operandos não tem sinal

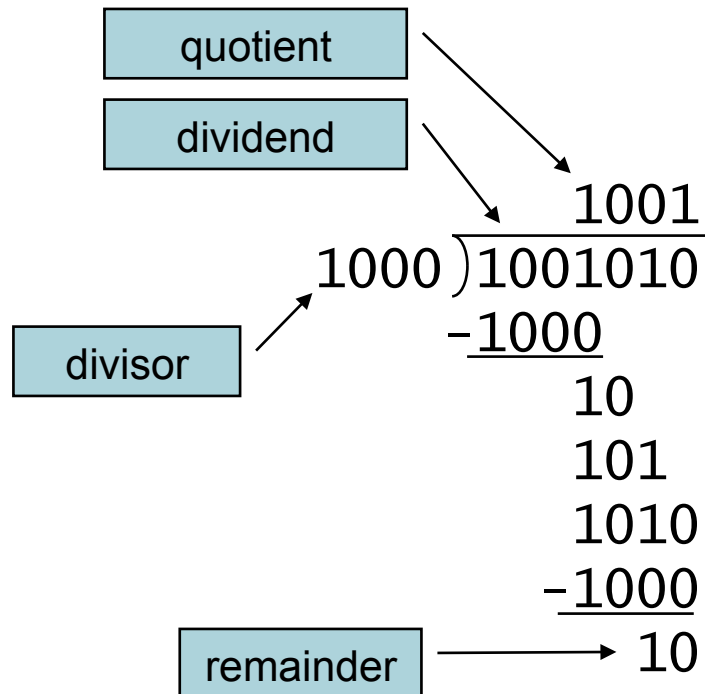
□ mulhsu: multiplica parte alta com e sem sinal

- Retorna os 32 bits mais significativos do produto, assumindo que um operando com sinal e outro sem

- Utiliza-se o mulh para checar *overflow* em 32 bits

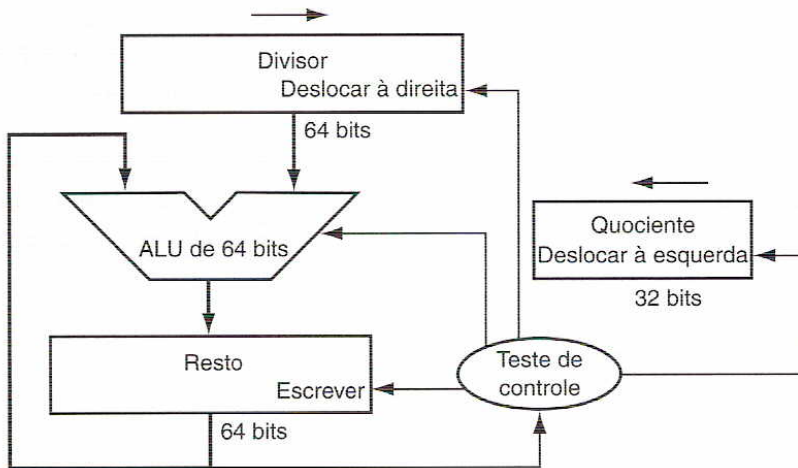


Divisão

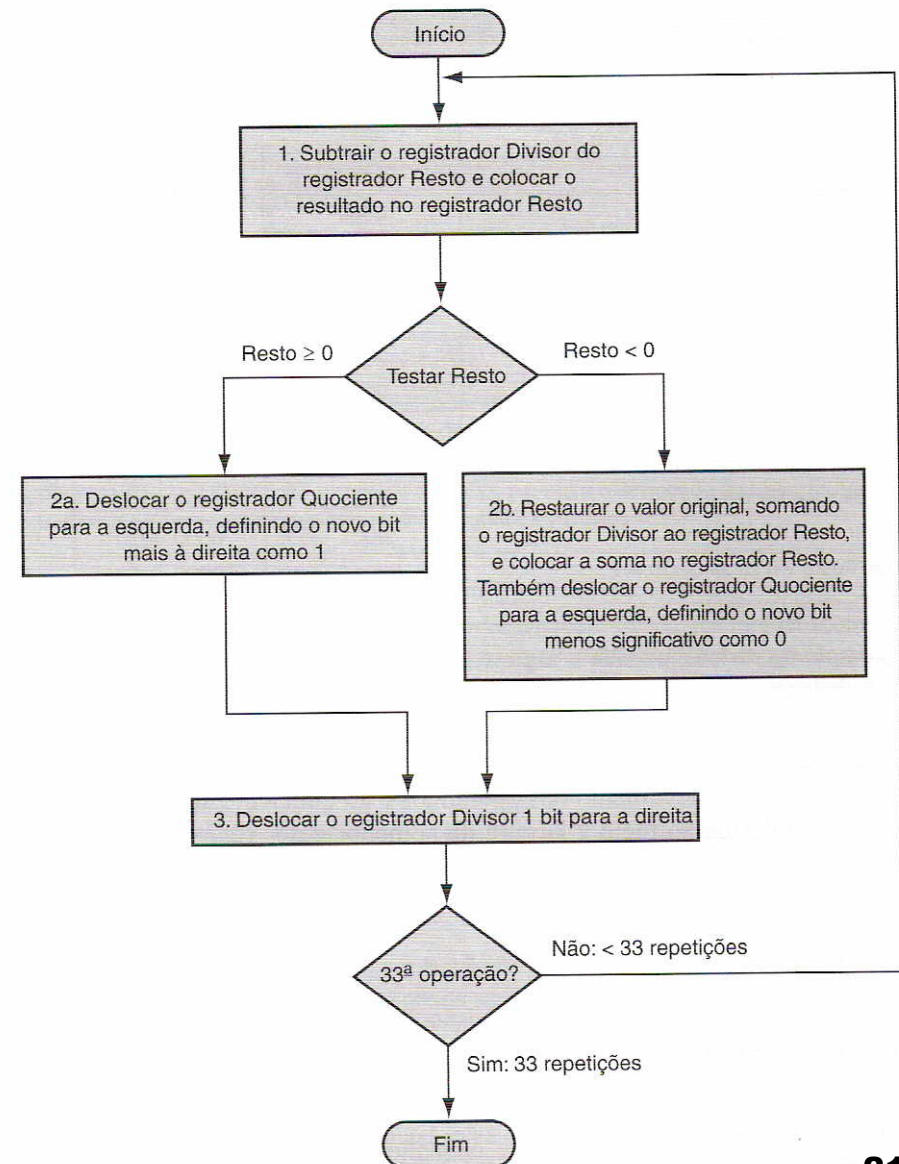


- Verificar divisão por zero
- Procedimento
 - Se divisor $\leq$ dividendo
 - 1 no quociente, subtrai
 - Senão
 - 0 no quociente, baixa próximo bit
- Restaura
 - Subtrai, se resto < 0 , soma de novo
- Divisão com sinal
 - Divide valores absolutos
 - Ajustar sinal do quociente e resto de acordo

Divisão Simples



Obs.: Existem versões otimizadas





Exemplo Divisão

Iteration	Step	Quotient	Divisor	Remainder
0	Initial values	0000	0010 0000	0000 0111
1	1: Rem = Rem – Div	0000	0010 0000	①110 0111
	2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0	0000	0010 0000	0000 0111
	3: Shift Div right	0000	0001 0000	0000 0111
2	1: Rem = Rem – Div	0000	0001 0000	①111 0111
	2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0	0000	0001 0000	0000 0111
	3: Shift Div right	0000	0000 1000	0000 0111
3	1: Rem = Rem – Div	0000	0000 1000	①111 1111
	2b: Rem < 0 $\Rightarrow$ +Div, sll Q, Q0 = 0	0000	0000 1000	0000 0111
	3: Shift Div right	0000	0000 0100	0000 0111
4	1: Rem = Rem – Div	0000	0000 0100	①000 0011
	2a: Rem $\geq$ 0 $\Rightarrow$ sll Q, Q0 = 1	0001	0000 0100	0000 0011
	3: Shift Div right	0001	0000 0010	0000 0011
5	1: Rem = Rem – Div	0001	0000 0010	①000 0001
	2a: Rem $\geq$ 0 $\Rightarrow$ sll Q, Q0 = 1	0011	0000 0010	0000 0001
	3: Shift Div right	0011	0000 0001	0000 0001



Multiplicação e Divisão com sinal

■ Na multiplicação:

- Basta verificar se os sinais do multiplicando e do multiplicador são iguais ou diferentes, definindo o sinal do produto.

■ Na divisão:

Precisamos definir o sinal do quociente e do resto.

- ☐ $\text{Dividendo} = \text{Quociente} \times \text{Divisor} + \text{Resto}$
- ☐ $7 \div 2 \rightarrow 7 = 3 \times 2 + 1$
- ☐ $(-7) \div 2 \rightarrow -7 = (-3) \times 2 + (-1) \quad \text{OU} \quad -7 = (-4) \times 2 + 1$

Regra:

Quociente: Mesma regra da multiplicação .

Resto: Mesmo sinal do Dividendo.



Divisão no RISC-V

■ Quatro instruções

- `div, divu` # quociente da divisão c/s sinal
 - Ex.: `div t0,t1,t2` # $t0 = \text{floor}(t1/t2)$
 - Não sinaliza erro em caso de divisão por zero!
- `rem, remu` # resto da divisão c/s sinal
 - Ex.: `rem t0, t1, t2` # $t0 = t1 \% t2$